PostgreSQL 16 — Advanced Indexing & EXPLAIN Plans

Source basis: PostgreSQL 16 official manual

Learning goals

- Explain core concepts and when they are useful.
- Read and adapt code/query examples.
- Recognize trade-offs, common errors, and verification steps.

How to use this chapter

Read the concept first, run examples in an isolated environment, then use the official references at the end of the chapter to verify version-specific behavior. The examples are teaching samples, not production-ready systems.

1. Scope and query-performance model

An index helps PostgreSQL locate rows without scanning every table row. Indexes can improve filtering, joins, ordering, and uniqueness enforcement, but add storage and write overhead.

Start tuning with a representative query, realistic data, and an execution plan. Do not add indexes based only on column names; measure the workload and revisit choices as data distribution changes.

This guide focuses on index types, composite and partial indexes, expression and covering indexes, index combination, and EXPLAIN plan interpretation.

Implementation and review notes

Use representative table sizes and current statistics. Compare estimated and actual rows, buffers, sorts, and loops. Treat index changes as hypotheses to test, not assumptions.

2. EXPLAIN and EXPLAIN ANALYZE

EXPLAIN shows the planner's chosen plan and estimated costs. EXPLAIN ANALYZE executes the statement and reports observed runtime statistics alongside estimates. Cost values are planner units, not milliseconds.

EXPLAIN ANALYZE on INSERT, UPDATE, or DELETE really executes the change. Use a transaction and roll back when safely testing data-changing statements. Read plans from leaf nodes upward and compare estimated rows with actual rows.

Example code / SQL

```
EXPLAIN
SELECT order_id, created_at
FROM orders
WHERE customer_id = 42
ORDER BY created_at DESC
LIMIT 20;

EXPLAIN (ANALYZE, BUFFERS)
SELECT order_id, created_at
FROM orders
WHERE customer_id = 42
ORDER BY created_at DESC
LIMIT 20;
```

Implementation and review notes

Use representative table sizes and current statistics. Compare estimated and actual rows, buffers, sorts, and loops. Treat index changes as hypotheses to test, not assumptions.

3. Sequential, index, and bitmap scans

A sequential scan reads table pages and checks rows. It can be optimal for small tables or queries returning a large share of rows. An index scan follows index entries to fetch matches.

A bitmap scan can combine index matches and visit heap pages in page-oriented order. PostgreSQL chooses plans based on estimated costs; an existing index is not guaranteed to be the best access path.

If estimates are inaccurate, inspect statistics, data skew, stale ANALYZE results, and predicates before changing planner settings.

Implementation and review notes

Use representative table sizes and current statistics. Compare estimated and actual rows, buffers, sorts, and loops. Treat index changes as hypotheses to test, not assumptions.

4. Index types

B-tree is the default general-purpose index and supports equality, range comparisons, and ordered retrieval for many types. Hash indexes support equality comparisons.

GIN suits values with multiple components, such as arrays, JSONB containment, and full-text search. GiST supports geometric, range, and other operator classes. SP-GiST supports partitioned search structures. BRIN summarizes ranges of table pages and can be compact for large physically correlated tables.

Choose an index type based on operators and operator classes used by the query.

Example code / SQL

```
CREATE INDEX idx_orders_customer_created
ON orders (customer_id, created_at DESC);

CREATE INDEX idx_events_payload_gin
ON events USING GIN (payload);

CREATE INDEX idx_logs_created_brin
ON logs USING BRIN (created_at);
```

Implementation and review notes

Use representative table sizes and current statistics. Compare estimated and actual rows, buffers, sorts, and loops. Treat index changes as hypotheses to test, not assumptions.

5. Composite indexes and key order

A multicolumn index stores keys in a defined order. For B-tree, leading columns are especially important: equality restrictions on leading columns followed by a range restriction can often narrow the scan efficiently.

Design key order around actual predicates and sorting. An index on (customer_id, created_at DESC) is a candidate for filtering one customer and returning recent orders, but not automatically optimal for filtering only by created_at.

Composite indexes consume storage/cache and increase INSERT, UPDATE, and DELETE work. Avoid creating every possible permutation.

Example code / SQL

```
CREATE INDEX idx_orders_customer_time
ON orders (customer_id, created_at DESC);

SELECT order_id, created_at
FROM orders
WHERE customer_id = 42
ORDER BY created_at DESC
LIMIT 20;
```

Implementation and review notes

Use representative table sizes and current statistics. Compare estimated and actual rows, buffers, sorts, and loops. Treat index changes as hypotheses to test, not assumptions.

6. Partial and expression indexes

A partial index stores only rows matching a predicate. It can reduce size and maintenance when queries repeatedly target a stable subset. PostgreSQL must be able to recognize that the query predicate implies the index predicate.

An expression index stores a computed expression, such as lower(email), and can speed lookups using the same expression. It adds write-time computation and storage.

Keep query expressions aligned with the index definition and verify the plan with EXPLAIN.

Example code / SQL

```
CREATE INDEX idx_jobs_ready
ON jobs (priority DESC, created_at)
WHERE status = 'ready';

SELECT job_id FROM jobs
WHERE status = 'ready'
ORDER BY priority DESC, created_at
LIMIT 50;

CREATE INDEX idx_users_lower_email ON users (lower(email));
SELECT user_id FROM users
WHERE lower(email) = lower('ada@example.com');
```

Implementation and review notes

Use representative table sizes and current statistics. Compare estimated and actual rows, buffers, sorts, and loops. Treat index changes as hypotheses to test, not assumptions.

7. Covering indexes and INCLUDE

A covering index stores enough columns to answer a query from the index. PostgreSQL supports INCLUDE columns on indexes such as B-tree. Included columns can be returned but do not participate in search ordering.

An index-only scan is possible when the index contains all required columns, but visibility-map state matters. PostgreSQL may still need heap fetches; selected columns being present does not guarantee zero heap visits.

Use INCLUDE sparingly because wider indexes consume storage/cache and increase write costs.

Example code / SQL

```
CREATE INDEX idx_orders_customer_cover
ON orders (customer_id, created_at DESC)
INCLUDE (total_amount, status);

SELECT created_at, total_amount, status
FROM orders
WHERE customer_id = 42
ORDER BY created_at DESC
LIMIT 20;
```

Implementation and review notes

Use representative table sizes and current statistics. Compare estimated and actual rows, buffers, sorts, and loops. Treat index changes as hypotheses to test, not assumptions.

8. Index combinations and ordering

PostgreSQL can combine indexes using bitmap AND/OR for some queries. This can help with selective independent filters, but bitmap construction and sorting may be slower than a well-designed composite index.

Indexes can support ORDER BY when their ordering matches the query. A LIMIT query may benefit because PostgreSQL can stop once enough rows are found. Unique indexes enforce uniqueness; primary keys create a unique index automatically.

Do not duplicate a primary-key or unique index without a measured reason.

Example code / SQL

```
CREATE INDEX idx_orders_status ON orders (status);
CREATE INDEX idx_orders_region ON orders (region);

EXPLAIN (ANALYZE, BUFFERS)
SELECT order_id FROM orders
WHERE status = 'paid' AND region = 'west';

-- Compare with a composite candidate:
CREATE INDEX idx_orders_status_region
ON orders (status, region);
```

Implementation and review notes

Use representative table sizes and current statistics. Compare estimated and actual rows, buffers, sorts, and loops. Treat index changes as hypotheses to test, not assumptions.

9. Statistics and common plan traps

The planner relies on statistics collected by ANALYZE and autoanalyze. If estimated rows differ substantially from actual rows, join choices and memory assumptions can be poor. Check statistics freshness and data distribution.

Common traps include testing tiny tables, comparing warm-cache and cold-cache runs, ignoring loops in nested-loop nodes, treating cost units as milliseconds, and trusting one execution as universal.

EXPLAIN (ANALYZE, BUFFERS) reports buffer hits/reads and can help diagnose I/O pressure. Be cautious running expensive statements in production.

Example code / SQL

```
ANALYZE orders;

EXPLAIN (ANALYZE, BUFFERS, VERBOSE)
SELECT c.customer_name, count(o.order_id)
FROM customers AS c
JOIN orders AS o ON o.customer_id = c.customer_id
WHERE o.created_at >= DATE '2026-01-01'
GROUP BY c.customer_name;
```

Implementation and review notes

Use representative table sizes and current statistics. Compare estimated and actual rows, buffers, sorts, and loops. Treat index changes as hypotheses to test, not assumptions.

10. Tuning workflow and exercises

Workflow: reproduce the query; capture EXPLAIN; use EXPLAIN ANALYZE with buffers where safe; compare estimated and actual rows; identify scans, joins, sorts, and loops; test one change; rerun under comparable conditions; document storage/write trade-offs.

Exercises: compare before/after a B-tree index; test a composite index for WHERE plus ORDER BY; test a partial index for a common status; and inspect whether INCLUDE enables an index-only scan.

Review questions: How does EXPLAIN ANALYZE differ from EXPLAIN? When might a composite index help? Why can an index-only scan still visit the heap?

Implementation and review notes

Use representative table sizes and current statistics. Compare estimated and actual rows, buffers, sorts, and loops. Treat index changes as hypotheses to test, not assumptions.

Official references and further reading

These trusted official sources are the authority for version-specific details. This PDF summarizes key ideas; consult the linked references for full API specifications and additional examples.

PostgreSQL 16: Indexes
https://www.postgresql.org/docs/16/indexes.html

PostgreSQL 16: Using EXPLAIN
https://www.postgresql.org/docs/16/using-explain.html

PostgreSQL 16: EXPLAIN command
https://www.postgresql.org/docs/16/sql-explain.html

PostgreSQL 16: CREATE INDEX
https://www.postgresql.org/docs/16/sql-createindex.html

PostgreSQL 16: Planner Statistics
https://www.postgresql.org/docs/16/planner-stats.html

Suggested practice checklist

- Confirm the runtime/version.
- Run the example in an isolated project or disposable database.
- Test one normal case and one failure/edge case.
- Record commands, expected output, and any assumptions.